



QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation

ANG LI, SAMUEL STEIN, SRIRAM KRISHNAMOORTHY, and JAMES ANG,

Pacific Northwest National Laboratory, USA

10

The rapid development of **quantum computing (QC)** in the NISQ era urgently demands a low-level benchmark suite and insightful evaluation metrics for characterizing the properties of prototype NISQ devices, the efficiency of QC programming compilers, schedulers and assemblers, and the capability of quantum system simulators in a classical computer. In this work, we fill this gap by proposing a low-level, easy-to-use benchmark suite called QASMBench based on the OpenQASM assembly representation. It consolidates commonly used quantum routines and kernels from a variety of domains including chemistry, simulation, linear algebra, searching, optimization, arithmetic, machine learning, fault tolerance, cryptography, and so on, trading-off between generality and usability. To analyze these kernels in terms of NISQ device execution, in addition to circuit width and depth, we propose four circuit metrics including gate density, retention lifespan, measurement density, and entanglement variance, to extract more insights about the execution efficiency, the susceptibility to NISQ error, and the potential gain from machine-specific optimizations. Applications in QASMBench can be launched and verified on several NISQ platforms, including IBM-Q, Rigetti, IonQ and Quantinuum. For evaluation, we measure the execution fidelity of a subset of QASMBench applications on 12 IBM-Q machines through density matrix state tomography, comprising 25K circuit evaluations. We also compare the fidelity of executions among the IBM-Q machines, the IonQ QPU and the Rigetti Aspen M-1 system. QASMBench is released at: <http://github.com/pnnl/QASMBench>.

CCS Concepts: • **Computer systems organization** → **Quantum computing**; • **Hardware** → **Quantum computation**; • **Software and its engineering** → **Software libraries and repositories**;

Additional Key Words and Phrases: Benchmark, OpenQASM, quantum metrics, NISQ

ACM Reference format:

Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Trans. Quantum Comput.* 4, 2, Article 10 (February 2023), 26 pages.

<https://doi.org/10.1145/3550488>

This material is based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, Co-design Center for Quantum Advantage (C2QA) under contract number DE-SC0012704. This research used resources of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725. We would like to thank the Pacific Northwest National Laboratory operated IBM-Q Hub. We also acknowledge support from Microsoft's Azure Quantum for providing credits and access to the ion-trap quantum hardware used in this paper. The Pacific Northwest National Laboratory is operated by Battelle for the U.S. Department of Energy under contract DE-AC05-76RL01830.

Authors' address: A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, Pacific Northwest National Laboratory, 902 Battelle Blvd, Richland, WA 99354, USA, 99354; emails: {ang.li, samuel.stein, sriram, ang}@pnnl.gov.

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of the United States government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2643-6817/2023/02-ART10 \$15.00

<https://doi.org/10.1145/3550488>

1 INTRODUCTION

Quantum computing (QC) [11, 98] has been envisioned as one of the most promising computing paradigms beyond Moore’s Law for tackling difficult computing challenges that are classically intractable arising from various domains like chemistry [48, 68], machine learning [15, 113], cryptography [49, 116], linear algebra [24, 57], finance [109, 134], recommendation systems [70], physics simulation [34, 42], networking [71, 94], and so on. QC relies on fundamental quantum mechanisms such as superposition, entanglement, interference, tunneling, etc. for performing computation, in hopes of significant or even exponential speedups over existing algorithms in classical computers [26, 116].

Despite holding great promise, QC in contemporary **noisy-intermediate-scale-quantum (NISQ)** [105] devices is still distant from outperforming classical computers regarding general problems. NISQ devices describe the near-term quantum platforms comprising 50 to hundreds of qubits, which is inadequate for full-scale error-correction but can already present promising results in a number of problems, and lay the foundations towards practical quantum demonstration [105]. The most recent public accessible NISQ device is IBM-Q Washington featuring 127 physical qubits [59].

The QC landscape is evolving rapidly nowadays. From the software perspective, large number of QC software are developed in classical programming languages (e.g., Python [51, 119], C/C++ [65], JavaScript [60], ML [5]). A list of opensource QC projects, quantum algorithms, and quantum simulators can be found in [43, 67, 107], respectively. From the hardware perspective, the development of QC testbeds have already proceeded to a stage where small working prototypes are available through cloud service, such as IBM-Q [59], Microsoft Azure [92], Amazon Braket [50], and Rigetti [110], showing encouraging results [22, 37]. Finally, from the application perspective, Google has announced the demonstration of quantum supremacy on the task of sampling the output of a pseudo-random quantum circuit, using a 53-qubit NISQ system [7]. Several quantum machine learning frameworks (e.g., [18, 60]) and chemistry simulation packages have been released as well (e.g., [60, 87]).

To close the gap between practical quantum applications and real quantum machines, contributions from the computer system and architecture community are undoubtedly critical in tackling several key challenges [21], including software and hardware verification [125], defining and perforating abstraction boundaries [96], managing parallelism and communication [79], mapping and scheduling operations [114], elevating control complexity [80], hardware-specific optimizations [95, 118, 126], investigating and mitigating noise [97, 124], and the like. These massive efforts, however, are currently evaluated and verified using randomly selected QC benchmarks [32, 79, 80, 95, 97, 114, 124–126], lacking the common ground for judicious analysis and fair comparison among each other. The communities thus urgently desire a low-level, easy-to-use QC benchmark suite to foster the software-hardware codesign effort for the purpose of validation and verification. This is particularly vital in the NISQ era, given the noise and technology limitation continuously to be the major challenges in the near future.

In this work, we propose a low-level benchmark suite called QASMBench based on the OpenQASM assembly-level **intermediate representation (IR)** [29]. It collects commonly used quantum algorithms and routines (e.g., the adder circuit in Figure 1) from a variety of distinct domains, including quantum chemistry, simulation, linear algebra, searching, optimization, arithmetic, machine learning, fault tolerance, cryptography, and so on. The design of the benchmark suite trades-off between generality and usability, covering a wide spectrum regarding circuit depth (i.e., number of gates) and width (i.e., number of qubits). Additionally, to analyze and compare the benchmark applications, we propose four circuit evaluation metrics including gate density, retention lifespan, measurement density, and entanglement variance, to offer deeper insights about the execution

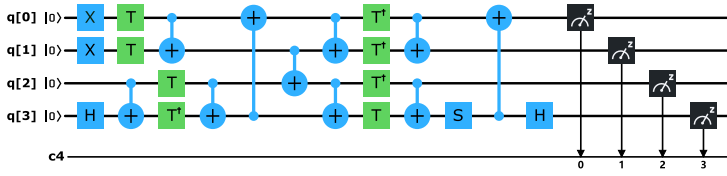


Fig. 1. A quantum adder circuit with width=4 (4 qubits) and depth=23 (23 gates) followed by measurement.

efficiency, the susceptibility to system/readout error, and the potential impact from device-specific optimizations, when mapping to a NISQ device. Circuits from QASMBench can be directly uploaded and evaluated in IBM-Q machines as well as other platforms (e.g., Rigetti through the Qiskit interface, IonQ and Quantinuum through Microsoft Azure Quantum [92]). Additionally, opensource tools such as *q-convert* [1] and *QCOR* [2] allow the translation of QASMBench to other QC programming languages or representations.

We evaluate QASMBench on 12 IBM-Q machines, and show the fidelity of circuit executions. The fidelity values are measured by calculating the Hellinger distance between the reconstructed density matrix of the resultant mixed state in a real quantum device through density matrix tomography, and the density matrix of the pure state by running the same circuit in a noiseless classical simulator. We observe that shallow circuits in general exhibit higher fidelity than deep circuits, likely due to less decay impact from a shorter execution time; whereas large quantum machines, such as the 127-qubit IBM-Q Washington, show reduced fidelity than small machines, despite using their best performing qubits. We also compare the fidelity of 4 QASMBench circuits among IBM-Q superconducting machines, IonQ’s QPU trapped-ion machine, and Rigetti’s Aspen M-1 superconducting machine, deriving interesting observations. Explanations are provided using the evaluation metrics proposed. Through such evaluation, we show how QASMBench and the six metrics can be used for evaluating contemporary and emerging NISQ platforms.

The remainder of the paper is organized as follows: In Section 2, we briefly describe NISQ and the landscape of OpenQASM. In Section 3, we introduce the QASMBench applications. In Section 4, we propose our characterization metrics. We characterize the QASMBench applications and evaluate them on IBM-Q and other NISQ devices in Section 5. We discuss related work in Section 6. Finally, we summarize and draw the conclusion in Section 7.

2 BACKGROUND: QUANTUM COMPUTING IN THE NISQ ERA

We describe NISQ devices and the OpenQASM programming environment in this section.

2.1 NISQ Devices

NISQ devices describe the near-term quantum platforms incorporating 50 to less than a thousand qubits [105]. The qubits are currently built using a variety of different technologies, including superconducting qubits [25, 111], trapped-ions qubits [23, 75], spin qubits [85, 104], photonic qubits [8, 99], and the like. To correctly run a QC circuit, the physical qubits have to stay coherent long enough for allowing the accomplishment of all the gate operations, thus imposing rigid constraints on allowable circuit depth [27]. The **T1 coherence** time describes the time for a qubit to decay from the excited state $|1\rangle$ to the ground state $|0\rangle$. The **T2 coherence** time describes the time for a qubit to transit its state due to environment interaction [126]. Consequently, gate operations should be quickly and precisely performed on the qubits during the coherent window for maintaining the quantum states. The probability of involving error during the gate operation is known as **gate error**. The measurement operation, which reads-out classical output from a qubit by collapsing its

Table 1. Basis Gates for IBM-Q, Rigetti, IonQ, and Quantinuum NISQ Devices

Vendor	Technology	1-qubit basis gates	2-qubit basis gates	Reference
IBM-Q	Superconducting	ID, RZ, SX, X	CX	[59]
Rigetti	Superconducting	RX, RZ	CZ, XY	[110]
IonQ	Trapped-Ion	GPI, GPI2, GZ	MS	[61]
Quantinuum	Trapped-Ion	RX, RZ	ZZ	[91]

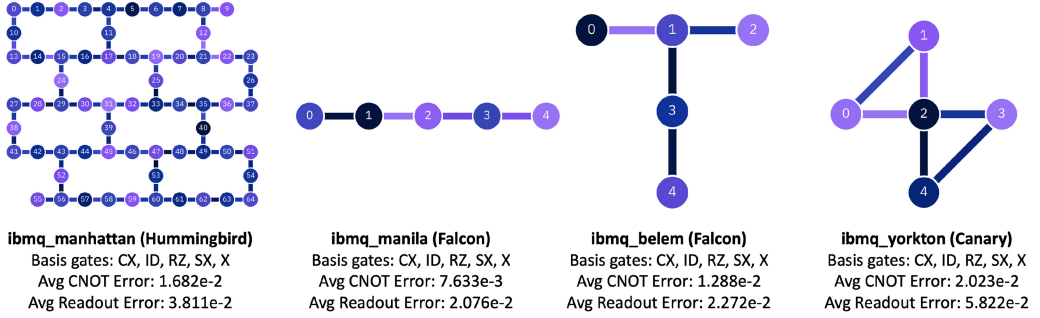


Fig. 2. IBM-Q device topology. The color of nodes implies frequency (GHz) of the qubit or how fast a 1-qubit gate can be executed. The connection color implies the gate time in nanoseconds for 2-qubit gates such as CX.

final quantum state, is also critical for the correctness and efficiency of QC [46]. The **readout error** describes the probability of inaccurate measurement of a qubit state [135]. To precisely sample the output distribution with the existence of gate and read-out error, a quantum circuit is often executed many times (known as **shots** [59] or repetitions [51]).

NISQ devices are mainly featured by two attributes at the system level: **basis gates** and **topology**. The basis gate set, also known as **instruction set architecture (ISA)** in classical computer architecture terminology, is hardware-defined. Upon execution, user quantum circuits are translated into an objective circuit only comprising basis gates. This process is called **quantum transpilation**. The basis gate sets for IBM-Q, Rigetti, IonQ and Quantinuum devices are listed in Table 1.

Additionally, due to technical limitations, it is possible that only certain qubits of a NISQ device are physically connected. In other words, the physical qubits in a NISQ device may follow a certain **topology** (also known as coupling). Figure 2 illustrates the topology of 4 IBM-Q devices. The topology limits the locations where two-qubit gates such as CX can be performed. If a CX is applied over two remote qubits, a series of SWAP gates are needed to relocate the two qubits to a connected tuple in the topology before the CX can be applied. For example, CX(0, 4) in *ibmq_manila* in Figure 2 requires 4 extra SWAP gates. Additionally, more swaps may be needed to switch the qubit(s) back to their original position(s), adding extra overhead and error. **Qubit mapping** describes the process of mapping of the logical qubits of a quantum circuit to the physical qubits of a NISQ device, which significantly impacts QC accuracy and execution efficiency.

2.2 OpenQASM Ecosystem

OpenQASM (Open Quantum Assembly Language) [29] is a low-level **intermediate representation (IR)** of quantum instructions, which is similar to traditional **Hardware Description Language (HDL)** like *Verilog* and *VHDL*. OpenQASM is a unified low-level assembly language for IBM-Q quantum machines. These NISQ devices, being accessible through IBM-Q cloud or IBM-Q

Table 2. OpenQASM Gate Definition (5 Basic Gates + 11 Standard Gates + 18 Composition Gates)

Gates	Meaning	Gates	Meaning	Gates	Meaning
U3	3 parameter 2 pulse 1-qubit	TDG	conjugate of sqrt(S)	CRZ	Controlled RZ rotation
U2	2 parameter 1 pulse 1-qubit	RX	X-axis rotation	CU1	Controlled phase rotation
U1	1 parameter 0 pulse 1-qubit	RY	Y-axis rotation	CU3	Controlled U3
CX	Controlled-NOT	RZ	Z-axis rotation	RXX	2-qubit XX rotation
ID	Idle gate or identity	CZ	Controlled phase	RZZ	2-qubit ZZ rotation
X	Pauli-X bit flip	CY	Controlled Y	RCCX	Relative-phase CXX
Y	Pauli-Y bit and phase flip	SWAP	Swap	RC3X	Relative-phase 3-controlled X
Z	Pauli-Z phase flip	CH	Controlled H	C3X	3-controlled X
H	Hadamard	CCX	Toffoli	C3XSQRTX	3-controlled sqrt(X)
S	sqrt(Z) phase	CSWAP	Fredkin	C4X	4-controlled X
SDG	conjugate of sqrt(Z)	CRX	Controlled RX rotation		
T	sqrt(S) phase	CRY	Controlled RY rotation		

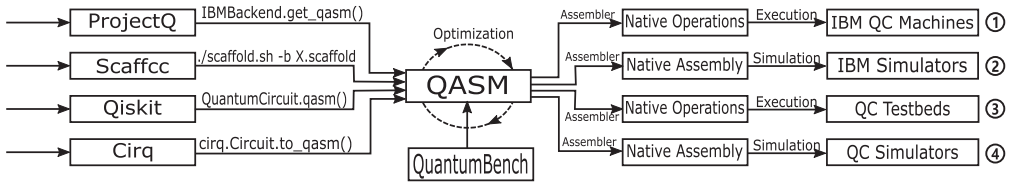


Fig. 3. The OpenQASM toolchain.

network [59], have been explored by many existing works [27, 58, 73, 95–97, 126]. An OpenQASM code can be directly uploaded and verified in an IBM-Q machine or launched through Qiskit.

Table 2 lists the types of gates that are defined in OpenQASM specification (i.e., the “qelib1.inc” header file) [29]. Within these gates, the first five, i.e., U3, U2, U1, CX, and ID, are *basic gates* that are expected to be supported by the quantum backend. From X to RZ are *standard gates* defined atomically in OpenQASM. These standard gates will be converted into basic gates during machine-specific assembling & mapping phase before actual execution. The remaining gates from CZ to C4X are *composition gates* that are constructed by standard gates. These gates are defined in `qelib1.inc` for the convenience of usage.

OpenQASM is a low-level assembly language. It is executed “sequentially” without any loops, branches, or jumps, making it very convenient for static analysis and simulating in a classical simulator. OpenQASM is widely supported. Several popular QC software frameworks such as Qiskit [60], Cirq [51], Scaffold [65], and ProjectQ [119] can dump the ready-to-execute circuit into an OpenQASM file, so it can be validated in the IBM-Q backends. Figure 3 shows the landscape of OpenQASM. As can be seen, OpenQASM stands at the conjunction between quantum software and hardware, being critical for both platform-agnostic or platform-specific optimization, mapping, scheduling, evaluation, profiling, and simulation. An OpenQASM based benchmark suite can be useful for all these activities. In the following, we briefly discuss each of the front-end frameworks.

Qiskit: The *Quantum Information Software Kit (Qiskit)* [60] is a quantum software platform developed by IBM. Qiskit is mainly based on Python but is also available in JavaScript and Swift [133]. The package comprises several tools, such as *qiskit-aer* for simulation, *qiskit-ignis* for hardware verification and noise addressing, and *qiskit-aqua* for exemplary applications. At present, Qiskit is probably the most widely used quantum programming language. Both Rigetti [110] and Microsoft Azure [92] (offering IonQ and Quantinuum system access) provide

Qiskit interface. OpenQASM can be easily dumped from a Qiskit program using “*QuantumCircuit.qasm()*”. Reversely, an OpenQASM circuit file can be loaded by Qiskit through “*QuantumCircuit.from_qasm_str()*”.

Cirq: Cirq [51] is a QC software platform developed by Google. It is based on Python. Despite claims to be the interface for connecting to their 72-qubit Bristlecone quantum computer, this is not yet available to the general users. Cirq incorporates a local simulator for generic gates, and a *Xmon-Simulator* for simulating the native gateset of Google’s quantum computers. In addition, Cirq offers a function called “*to_qasm()*” in each circuit object so that a circuit can be dumped as an OpenQASM code that is runnable on IBM-Q backends. Note, based on our experience, not all Cirq code can be translated to OpenQASM.

Scaffold: Scaffold [65] is a quantum programming language embedded in the C/C++ programming language based on the LLVM compiler toolchain [74]. The major goal of Scaffold is to assist in developing quantum algorithms and advanced optimization, leveraging the existing LLVM compiling flow. By supporting advanced language structures such as loops, Scaffold can generate very complex OpenQASM code, such as the VQE examples in QASMBench. A Scaffold program can be compiled by *Scaffcc* [65] to native OpenQASM code using the “*-b*” compiler option.

ProjectQ: ProjectQ is a quantum software platform developed by Steiger et al. from ETH Zürich [119]. Similar to Scaffold, ProjectQ does not have its own dedicated real quantum backend, but relies on classical simulation. However, it provides a way to generate OpenQASM code so that a ProjectQ program can be verified on an IBM testbed — through “*IBMQBackend.get_qasm()*”.

3 QASMBENCH

We introduce QASMBench in this section. QASMBench is an end-to-end package comprising a diverse variety of benchmark circuits, a system for evaluating the performance of a circuit, and characterizing circuit metrics for interpreting circuit characteristics both pre and post transpilation to real machines. Some of the benchmark routines are generated and reproduced from existing open-source QC software packages, including [29, 51, 60, 65, 87, 90, 112] through the approaches described in Figure 3, whereas others are locally developed.

Depending on the number of qubits used, QASMBench is partitioned into three categories:

- **Small-scale**, with qubits ranging from 2 to 5. The purpose is to allow intensive measures such as density matrix tomography, with limited cost. Also, the present IonQ QPU and most IBM-Q devices with public access feature 5 qubits. The benchmarks in this category are listed in Table 3.
- **Medium-scale**, with qubits ranging from 6 to 15, for general benchmarking usage. Another historical reason is that when QASMBench was firstly developed, the publicly accessible IBM-Q machine with the maximum number of qubits was *ibmq_melbourne*, which has 15 qubits. However, *ibmq_melbourne* is already retired now.
- **Large-scale**, contains benchmarks with more than 15 qubits, as listed in Table 5. The biggest circuit in QASMBench is *adder_n127* that contains 3,991 gates. This corresponds to the most recent IBM-Q machine – *ibmq_washington* which features 127 physical qubits.

For each item in Table 3–5, we list its name, brief description, and the algorithm category it belongs to in the quantum stack (see Figure 4). We show the number of qubits and gates utilized in the routines. The gate number here refers to *standard* OpenQASM gates (not basic gates or composition gates) as discussed in Section 2.2. We also list the number of CX gates in the tables, which indicates a major source of delay and error. Each algorithm is briefly introduced under its respective category. However, certain routines appear in multiple categories, but we only describe

Table 3. QASMBench Small-scale Benchmark

Benchmark	Description	Algorithms	Qubits	Gates	CX	Ref
adder	Quantum ripple-carry adder	Quantum Arithmetic	4	23	10	[65]
basis_change	Transform the single-particle basis of a linearly connected electronic structure	Quantum Simulation	3	53	10	[87]
basis_trotter	Implement Trotter steps for molecule LiH at equilibrium geometry	Quantum Simulation	4	1626	582	[87]
bell_state	Bell State	Logical Operation	2	3	1	[131]
cat_state	Cat State	Logical Operation	4	4	3	[76]
deutsch	Deutsch algorithm with 2 qubits for $f(x) = x$	Hidden Subgroup	2	5	1	[29]
dnn	Quantum Deep Neural Network	Quantum Machine Learning	2	268	84	[121]
fredkin_n3	Fredkin gate benchmark	Logical Operation	3	19	9	[102]
qec_dist3	Error correction with distance 3 and 5 qubits	Error Correction	5	114	49	[90]
grover	Grover's algorithm	Search and Optimization	2	16	2	[6]
hs4	Hidden subgroup problem	Hidden Subgroup	4	28	4	[65]
inverseqft	Performs an exact inversion of quantum Fourier transform	Hidden Subgroup	4	8	0	[29]
iSWAP	An entangling swapping gate	Logical Operation	2	9	2	[65]
linearsolver	Solver for a linear equation of one qubit	Linear Equation	3	19	4	[16]
lqn	Learning parity with noise	Machine Learning	5	11	2	[112]
pea	Phase estimation algorithm	Hidden Subgroup	5	98	42	[29]
qaoa	Quantum approximate optimization algorithm	Search and Optimization	3	15	6	[62]
qec_sm	Repetition code syndrome measurement	Error Correction	5	5	4	[29]
qec_en	Quantum repetition code encoder	Error Correction	5	25	10	[112]
qft	Quantum Fourier transform	Hidden Subgroup	4	36	12	[29]
qrng	Quantum Random Number Generator	Quantum Arithmetic	4	4	0	[29]
quantumwalks	Quantum walks on graphs with up to 4 nodes	Quantum Walk	2	11	3	[89]
shor	Shor's algorithm	Hidden Subgroup	5	64	30	[60]
toffoli	Toffoli gate	Logical Operation	3	18	6	[65]
teleportation	Quantum Teleportation	Quantum Communication	3	8	2	[41]
jellium	Variational ansatz for a Jellium Hamiltonian with a linear-swap network	Quantum Simulation	4	54	16	[87]
vqe_uccsd	Variational Quantum Eigensolver with UCCSD ansatz	Search and Optimization	4	220	88	[53]
wstate	W-state preparation and assessment	Logical Operation	3	30	9	[29]

Table 4. QASMBench Medium-scale Benchmarks

Benchmark	Description	Domain	Qubits	Gates	CX	Ref
dnn	Quantum Deep Neural Network	Quantum Machine Learning	8	1200	384	[121]
adder	Quantum ripple-carry adder	Quantum Arithmetic	10	142	65	[29]
bb84	A quantum key distribution circuit	Quantum Communication	8	27	0	[51]
bv	Bernstein-Vazirani Algorithm	Hidden Subgroup	14	41	13	[29]
ising	Ising model simulation via QC	Quantum Simulation	10	480	90	[65]
multiplier	Quantum multiplier	Quantum Arithmetic	15	574	246	[51]
multiply	Performing 3×5 in a quantum circuit	Quantum Arithmetic	13	98	40	[6]
qaoa	Quantum approximate optimization algorithm	Search and Optimization	6	270	54	[51]
qf21	Using quantum phase estimation to factor the number 21	Hidden Subgroup	15	311	115	[6]
qft	Quantum Fourier transform	Hidden Subgroup	15	540	210	[29]
qpe	Quantum phase estimation algorithm	Hidden Subgroup	9	123	43	[6]
sat	Boolean satisfiability problem via QC	Searching and Optimization	11	679	252	[29]
seca	Shor's error correction algorithm for teleportation	Error Correction	11	216	84	[6]
simons	Simon's algorithm	Hidden Subgroup	6	44	14	[6]
vqe_uccsd	Variational quantum eigensolver with UCCSD	Linear Equation	8	10808	5488	[65]

them in the first appearance. Last but not the least, although QASMBench is developed based on OpenQASM, they can be converted to other representations such as Q#, PyQuil, Cirq, etc. through the Javascript based *q-convert* tool, which is available online: <http://quantum-circuit.com/qconvert>. We distribute QASMBench in other representations as well through a modified version of *q-convert*.

3.1 Small-scale Benchmarks

Adder: The quantum adder uses each qubit as a classical bit, and builds a traditional adder through quantum gates. Here, we build the full-adder circuit using a basic component — a 4-qubit ripple-carry adder [31, 129] with a carry-in and a carry-out qubit. Given the granularity of 4-qubits, a m -qubit adder would require $2m + m/4 + 1$ qubits in total where $m/4 + 1$ qubits are used for the carrying bits between the basic 4-qubit adder components. We included a 4-qubit adder (see Figure 1) in small-scale, a 10-qubit adder in medium-scale, and 18-/127-qubit adders in large-scale. This benchmark is motivated by its scalability, easy-to-verify and usefulness in building quantum numeric units in the post-NISQ era.

Basis_change: The OpenQASM routine of this benchmark is generated from Google's Cirq based OpenFermion library [87] for manipulating fermionic systems towards chemistry simulation on

Table 5. QASMBench Large-scale Benchmarks

Benchmark	Description	Domain	Qubits	Gates	CX	Ref
adder	Quantum ripple-carry adder	Quantum Arithmetic	127	3991	910	[65]
bigadder	Quantum ripple-carry adder	Quantum Arithmetic	18	284	130	[29]
bv	Bernstein-Vazirani algorithm	Hidden Subgroup	19	56	18	[29]
cat_state	Cat State	Logical Operation	100	100	99	[76]
cc	Counterfeit coin finding problem via QC	Hidden Subgroup	18	34	17	[29]
ghz_state	GHZ State preparation and assessment	Logical Operation	23	23	22	[52]
ising	Ising model simulation via QC	Quantum Simulation	26	280	50	[65]
ising	Ising model simulation via QC	Quantum Simulation	60	654	118	[65]
multiplier	Quantum multiplier	Quantum Arithmetic	25	3723	750	[51]
qft	Quantum Fourier transform	Hidden Subgroup	20	970	380	[29]
qft	Quantum Fourier transform	Hidden Subgroup	85	17935	7140	[29]
square_root	Computing the square root of a number via amplitude amplification	Quantum Arithmetic	18	4640	898	[65]
swap_test	SWAP Test algorithm implementation	Logical Operation	25	446	96	[69]
wstate	W-state preparation and assessment	Logical Operation	27	446	96	[29]

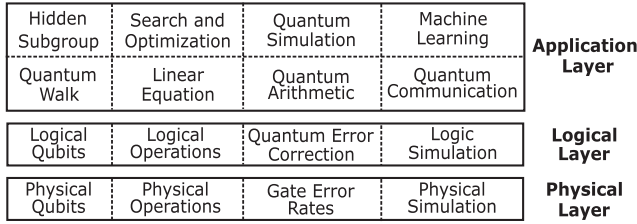


Fig. 4. Quantum computing stack for the categorization of QASMBench. It is built based on [93] by augmenting the categories of quantum arithmetic, quantum machine learning, and quantum communication.

quantum computers. It shows how to use a quantum circuit to transform the single-particle basis of an linearly connected electronic structure so as to realize exact evolution under a random Hermitian one-body fermionic operator. This is included as a quantum chemistry benchmark.

Basis_trotter: This routine is also generated from the Cirq based OpenFermion library [87] using the method described in Figure 3. It is designed to implement the Trotter [14, 81] steps for an actual molecule LiH at equilibrium geometry, so as to simulate basis molecular Hamiltonians taking the following form $H = \sum_{pq} T_{pq} a_p^\dagger a_q + \sum_{pqrs} V_{pqrs} a_p^\dagger a_q^\dagger a_r a_s$. The circuit uses 4 qubits but is quite deep, challenging the T1 and T2 decay of a NISQ device. It is included as another chemistry benchmark.

Bell State: The Bell state describes the Bell Inequality Test. This algorithm implements the Bell experiment on a quantum computer. This is a fundamental quantum mechanics experiment, and consists primarily of local entanglement, and low SWAP cost no matter the topology.

Cat state: The Cat-state is an imperative quantum sub routine, and forms the basis of quantum networking's EPR pairs [56]. The circuit comprises entangling n qubits, where the readout bitstring of the algorithm is homogeneous in 0 or 1 (i.e. 00...0 or 11...1). The bell state challenges primarily the entangling capabilities of a system.

Deutsch: The Deutsch-Jozsa algorithm [33] is a generalization of Deutsch's algorithm. The problem is defined as – given an oracle function $f\{0, 1\}^n \rightarrow \{0, 1\}$ which takes an n binary digit as input, and outputs either 0 or 1. The objective is to find whether the outputs are the same for all of the inputs (say constant), or are one value (i.e., 0 or 1) for half of the inputs and the other (i.e., 1 or 0) for the remaining half (say balanced). Here, if the 2^n possible inputs are encoded into an n -qubit register, and if the oracle function can be expressed as gate operations in a black-box, the quantum computer can get the answer with only one-time evaluation to the oracle function whereas a deterministic classical computer would require $2^{n-1} + 1$ times evaluations. This algorithm is among

the first examples to demonstrate that a quantum algorithm can achieve exponentially speedup over any possible deterministic classical algorithms. It is included as a baseline circuit.

DNN: Quantum **deep neural networks** [121] is a quantum adaption of the classical neural network where variational quantum circuits with a layered-approach are parameterized by a bank of θ values. **QNN** consists of gates such as $\text{RY}(\theta)$, CX and $\text{CRY}(\theta)$ for encoding classical data and weight information as well as introducing entanglement. Tasks such as classification are commonly used in evaluating these models where an attainable objective function is described alongside quantum gate differentiation, where the circuit is optimized. The domain of **Quantum Machine Learning (QML)** and **QNN** have drawn substantial attention recently [18, 122] as the hope for a quantum advantage in machine learning, thereby motivating the inclusion of the quantum DNN benchmark.

Fredkin: The Fredkin operation in Quantum Computing forms the foundation for the SWAP test, an important machine learning operation [122]. Fredkin gates challenge topologies by interacting three qubits with each other.

QEC: Quantum error correction (QEC) is to introduce redundancy into the original circuit by using additional physical qubits so that the state of the original circuit is effectively protected against noise. The surface code is often considered to be among the most promising QEC approaches as it can cope with error rates around 10^{-2} [45]. The *qec_dist3* benchmark performing a 3-distance 5-qubit surface code which is developed based on [90]. It is the smallest code that can correct arbitrary single-qubit error. QASMBench includes other QEC benchmarks, such as **qec_en** and **qec_sm** for encoder and syndrome measurement of repetition code, and **seca** for Shor's QEC for teleportation. QEC is perhaps the most important algorithm for NISQ and the success of QC in the long run, hence we include four QEC benchmarks in QASMBench.

Grover: Grover's algorithm [54] is a quantum algorithm for searching in a database, offering quadratic speedup over classical searching methods. It takes a black-box oracle realizing the function: $f(x) = 1$ if $x = y$, $f(x) = 0$ if $x \neq y$, and find y within a randomly ordered sequence of N items using $O(\sqrt{N})$ operations and $O(N \log N)$ gates with probability $p \geq 2/3$. Grover's algorithm is appealing in that it determines with high probability the unique input given the output is pre-known. Grover's algorithm is one of the fundamental quantum algorithms considering the importance of searching in the big-data era.

HS4: Hidden subgroup problems [65] is a quantum problem capturing problems such as factoring and graph isomorphism. The algorithm is important in the theory of Shor's algorithm, and hence plays an important role in the evolution of benchmarking quantum computers.

QFT & InverseQFT: The **Quantum Fourier Transform (QFT)** [26] (and its inverse) applies (inverse) Fourier transformation to the wave function amplitudes. It is a linear transform over the states of qubits, which is the quantum analogue of the discrete (inverse) Fourier transform. QFT is a basic component of many well-known quantum algorithms, including Shor's algorithm, quantum phase estimation, hidden subgroup problem, and the like. Besides, it has some interesting features such as zero entanglement variance, as will be discussed later.

iSWAP: The iSWAP operation is a challenging dual qubit operation comprising multiple CNOT operations as well as single qubit operations. It is well suited to challenge two local qubits on their gate performance.

LinearSolver: This benchmark realizes the HHL solver [57] for solving a linear equation of 1-qubit. The HHL algorithm estimates the outcome of a scalar measurement on the solution vector to a sparse linear equation system. We originally attempted to dump the multi-qubits HHL routine from Cirq [51], but encountered error in generating the OpenQASM code due to the mismatch between Google's native quantum IR and OpenQASM (e.g., missing the CCCRy representation). HHL is a

critical quantum algorithm given the importance of linear algebra in scientific computation and machine learning.

LPN: The **LPN (learning parity with noise)** benchmark is a machine learning problem of learning a hidden parity function defined by the unknown binary string in the presence of noise. This application is very promising for NISQ devices because (i) it is computationally intractable for classical approaches [101]; and (ii) it actually leverages the noise presented in the NISQ devices [30], which is usually seen as a hurdle in other applications.

Pea: Quantum phase-estimation (QPE) [98] is an algorithm to compute the eigenvalue $e^{2\pi i\theta}$ of a unitary operator operating on m qubits with the eigenvector $|\psi\rangle$ such that $U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$, $0 \leq \theta < 1$. QPE is another fundamental quantum algorithm being the key component of Shor's method. This *Pea* benchmark circuit describes a 2-qubit system including a read-out ancilla bit, and a physical system bit [35], showcasing the property of QPE.

QAOA: The **quantum approximate optimization algorithm (QAOA)** [39, 136] is another variational quantum algorithm that is particularly interesting to NISQ devices, given its ability to tolerate a certain degree of noise through the unique quantum-classic hybrid algorithm design. QAOA is designed to solve combinatorial optimization problems [135], such as in graph analytics. In QAOA, a quantum subroutine is embedded in a classical search loop. The initial quantum state is prepared according to a set of variational parameters. These parameters are adjusted based on the measurement in the previous iteration. The QAOA benchmark in QASMBench represents the quantum workload for a sample iteration.

QuantumWalks: Quantum walks [4, 130] is the quantum analogue of the classical random-walk algorithm, which has been adopted for many problems including graph isomorphism [47] and ranking nodes in a network [100]. Both discrete and continuous-time quantum-walk algorithms have been formulated by existing works [4, 40]. The quantum-walk benchmark here represents the workload for a single step of the walks [89].

QEC SM & EN: Quantum Error correction codes are examples of current approaches to quantum error correction. These circuits form the foundation for Quantum Error Correction, and can be used to gain intuition as to the processors' performance for error correction.

QRNG: Quantum random number generator is an algorithm for generating completely random numbers. It is known that no absolutely random random number generator exists, and is a relatively simple quantum circuit with no entanglement. This routine, although simple, can be very useful for real world applications.

Shor: Shor's algorithm [115] is a famous QC algorithm for integer factorization in polynomial-time. It demonstrates theoretical exponential speedups over the classical algorithms, posing substantial threat and concern over the widely-used public-key cryptography systems like RSA. Given its importance in quantum cryptography, we include a 5-qubit sample here in QASMBench. We also include another example — **qf21** for showcasing the process of factoring the integer of 21.

Toffoli: Similar to the Fredkin gate, the Toffoli gate [127] or CCX) gate is another universal gate. It refers to a circuit with three inputs and three outputs that inverts the third qubit if the first two qubits are both 1, otherwise all bits stay unchanged. We include the 3-qubit Toffoli gate as a baseline benchmark in QASMBench.

Teleportation: The teleportation routine is a quantum communication channel connecting two qubits, communicating through an ancilla third. Quantum teleportation challenges a processors' ability to communicate information between an ancilla qubit.

VQE: The **variational quantum** algorithm is to optimize a parameterized quantum circuit ansatz applied to some initial state for minimizing a cost function defined according to the output state

[66, 88, 120]. When applied in quantum simulation (i.e., simulating physical phenomena using QC rather than simulating QC in a classical computer), the goal of the algorithm is often to prepare ground states. The cost function is often the expectation value of a Hamiltonian. If the initial state is $|\psi\rangle$, the Hamiltonian is H , the ansatz is $U(\vec{\theta})$ where $\vec{\theta}$ is the variational parameter, then the cost function is $E(\vec{\theta}) = \langle \psi | U^\dagger(\vec{\theta}) H U(\vec{\theta}) | \psi \rangle$. The *Jellium* benchmark creates a variational ansatz for a Jellium Hamiltonian via a linear-swap network. We also include two *Unitary Coupled Cluster Single-Double* VQE ansatz [132] for estimating the ground energy of molecules due to their ultra deep circuits.

W_state: *w_state* is one of the basic states exhibiting properties not observed in a classical system. The *w_state* [38] is a special type of entangled quantum state generalized by:

$$|W\rangle = \frac{1}{\sqrt{n}}(|10\dots 0\rangle + |01\dots 0\rangle + \dots + |00\dots 1\rangle) \text{ for } n > 2$$

It describes the condition that for an n -qubit system, no matter which single qubit is measured or lost, the remaining $(n-1)$ qubits still remain entangled. This property makes it useful for improving the robustness of ensemble-based quantum memories [44]. This routine implements the 3-qubit condition where the three qubits follow $|W\rangle = \frac{1}{\sqrt{3}}(|001\rangle + |010\rangle + |100\rangle)$.

3.2 Medium-scale Benchmarks

BB84: *BB84* [12] is a **quantum key distribution (QKD)** protocol which is the first quantum cryptographic protocol [17] based on the non-cloning quantum laws for showcasing provably secure key generation. It relies on the fact that it is not possible to obtain information distinguishing two non-orthogonal states without disturbing the signal. Since QKD is a critical component for quantum communication and quantum networks, we include BB84 as a sample benchmark in QASMBench.

BV: The **Bernstein-Vazirani** algorithm [13] is an extension to the Deutsch-Josza algorithm [33]. The problem is that, given a black-box function f , which takes in n bits as input, and outputs the bitwise product of a string s against the input. This is defined as $x, f(x) = s \times x \bmod(2)$ where the string s is hidden. Classically, to determine the hidden string, we need n evaluations $(100\dots 0, 0100\dots 0, \dots, 0000\dots 1)$. This algorithm can decide s through a single query to the oracle on a quantum computer. Given that it is another application that can showcase exponential quantum speedup, we include it in QASMBench.

Ising: The Quantum Ising Model [19, 72] consists of an array of quantum spins arranged in a certain lattice. The spins, which are expressed in quantum operators, can only interact with their neighbors. The phase transition is due to quantum fluctuation introduced by the transverse field. Given the quantum annealing machines (e.g., D-Wave) basically resolving optimization problems that can be expressed in an Ising model, we include it as a benchmark routine in QASMBench.

Multiplier: Similar to the adder, the quantum multiplier also uses a qubit as a classical bit to construct a multiplier unit. Our implementation is similar to the one from Cirq [51], which requires $5n$ qubits for an n -qubit multiplier and internally calls the n -qubit adder by n times for aggregating the partial results. The multiplier comprises the three phases: *preparing operand*, *multiply*, and *measure*. Figure 5 shows an example of a 2-qubit multiplier using 10 qubits. Similarly to the adder, this is motivated by its simple scalability and easy verification.

SAT: The **Boolean Satisfiability problem (SAT)** determines whether there is an assignment of variables that satisfies a given Boolean function [9, 123] which is one of the first proven NP-complete problems. Since many optimization problems (e.g., in power-grid domain) can be attributed as a **mixed-integer-linear-programming (MILP)** problem, which can be formulated as a

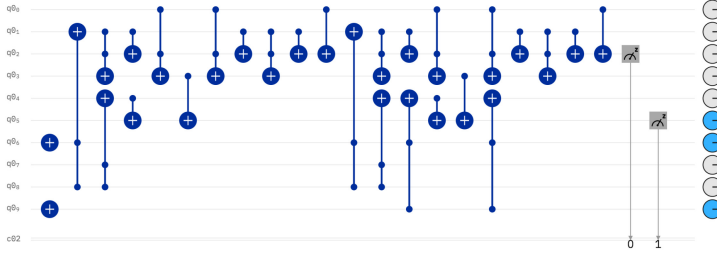


Fig. 5. A 2-qubit multiplier. The execution phases include: (1) *Preparation* (2) *Execution* (3) *Measurement*.

binary optimization problem [20] similar to SAT, we include SAT here as a benchmark in QASM-Bench.

Simon: Simon’s algorithm [117] was among the first quantum algorithms to show exponential speedups over the best classical deterministic or probabilistic algorithm. It finds an unknown nonzero $s \in \{0, 1\}^n$ that satisfies $f(x) = f(x \oplus s)$. We include it here as another benchmark showing quantum advantage. Besides, Simon’s algorithm is more sensitive to noise in NISQ devices.

3.3 Large-scale Benchmarks

CC: The *counterfeit coin* problem is a mathematical puzzle that attempts to find all false coins from a given bunch of coins using a balance scale [63]. Let’s say you have a set of N coins, and up to k of them are counterfeit and hence lighter. Using a scale, which returns the information of “balanced” or “tilted”, we can deduce if any, or how many of the coins are counterfeit. The classical algorithm requires a time complexity of $O(k \log(k/N))$. The quantum algorithm offers an exponential speed-up, with a time complexity of $O(k^{\frac{1}{4}})$ [64].

GHZ-state: A *Greenberger-Horne-Zeilinger* (GHZ) state [52] is a particular type of entangled quantum state that involves at least three qubits: $|GHZ\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$. Since it describes the superposition of all qubits being 0 with all of them being 1, it represents the maximally entangled quantum state. Consequently, characterizing GHZ state can provide very useful information about the fidelity of multi-qubit interactions, which is critical for constructing large-scale quantum computers. For an n -qubit system, GHZ state is generalized as:

$$|GHZ\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle^{\otimes n}) \text{ for } n > 2$$

To construct a GHZ-state for an n -qubit system, from an all-zero states $|\Psi\rangle = |0\rangle^{\otimes n}$, after an H gate is applied on the first qubit, $|\Psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle \otimes |0\rangle^{\otimes n-1})$. We then apply a series of CX (or CNOT) gates from qubit 0 to qubit $(n-1)$. Running on a full-fidelity quantum machine with 5 logic qubits, the measurement result should be 50% $0^{\times n}$ and 50% $1^{\times n}$. This application represents another basic quantum state which exploits across all the available qubits. It can be used to prove CNOT gate stability of a NISQ device.

Squarer_root: This routine relies on amplitude amplification [55] to find the square root of an n -bit number using Grover’s search technique. This is a complex application in QASMBench.

Swap-test: The swap-test is a very convenient and economic way to measure the difference between two quantum states, or how two quantum ($|\psi\rangle$ and $|\phi\rangle$) states are divergent from each other. The swap-test can be very useful in quantum machine learning for estimating the prediction accuracy (e.g., distance of prediction from the tag). It uses an ancilla qubit for example qubit 0. The initial state of the system is $|0, \psi, \phi\rangle$. The ancilla qubit is firstly put in superposition via an H gate,

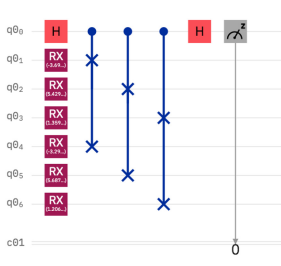


Fig. 6. 3-qubit swap_test.

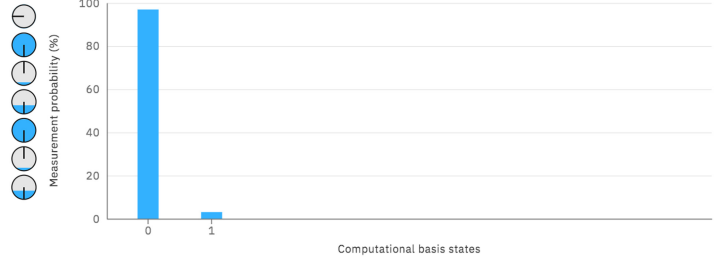


Fig. 7. Measurement result for a 3-qubit swap_test circuit using 7 qubits.

as shown in Figure 6, which converts the system state into $\frac{1}{\sqrt{2}}(|0, \psi, \phi\rangle + |1, \psi, \phi\rangle)$. By applying the series of controlled-swap gate (or CSWAP, which swaps the states of two qubits only if the control qubit is 1, see Table 2), the system state becomes $\frac{1}{\sqrt{2}}(|0, \psi, \phi\rangle + |1, \phi, \psi\rangle)$. Then, by applying another H gate on qubit-0, the system transits to $\frac{1}{2}(|0\rangle(|\psi, \phi\rangle + |\phi, \psi\rangle) + |1\rangle(|\psi, \phi\rangle - |\phi, \psi\rangle))$. When measuring qubit-0, the probability of obtaining 0 is $P(0) = \frac{1}{2} + \frac{1}{2}|\langle\psi|\phi\rangle|^2$. Therefore, if $|\psi\rangle$ and $|\phi\rangle$ are orthogonal, $|\langle\psi|\phi\rangle|^2 = 0$, the probability of obtaining 0 is 50%; if $|\psi\rangle$ and $|\phi\rangle$ are equal, $|\langle\psi|\phi\rangle|^2 = 1$, the probability of obtaining 0 is 100%. The more the two states are similar, the higher the probability of obtaining 0 approaches 1. Note that the swap-test needs sufficient number of repetitions to achieve an accurate sampling of the probability of the ancilla qubit. The measurement probability in Figure 7 shows that the two 3-qubit states in Figure 6 are similar to each other with $P(0) \approx 1$. The SWAP test is a crucial algorithm in many Quantum processes, and is easily scaled out to any application size with substantial entanglement, hence is an ideal NISQ evaluation benchmark.

4 QASMBENCH CHARACTERIZATION METRICS

We propose a set of quantum circuit based evaluation metrics representing various features of a quantum application. These metrics are designed such that through them certain estimation can be performed on executing a particular circuit over a particular NISQ device, practicing the software-hardware co-design paradigm. The metrics serve as useful indicators on how a quantum circuit can stress a NISQ hardware device. The scripts for profiling these metrics are provided along with the QASMBench code.

4.1 Circuit Width

Circuit width is defined as the number of qubits that enter the superposition state at least once within an application's lifespan. Qubits that are measured in the interim of a circuit and re-enter superposition are only counted as one qubit towards the circuit width. Circuit width dictates the spatial capacity required for a quantum device in order to run the quantum circuit. Circuit Width is mathematically defined in Equation (1) where $n_{q_{\text{active}}}$ is the number of qubits that are in-use or demanded by the circuit.

$$\text{Circuit Width} = n_{q_{\text{active}}} \quad (1)$$

4.2 Circuit Depth

Circuit depth is defined as the minimum time-evolution steps required to complete a quantum application. Time evolution is the process of completing all gates defined at time $t = t_j$, and once these are completed, the circuit moves onto time $t = t_{j+1}$, where the following gates are to be processed. To keep generality and avoid the impact from low-level optimization, circuit depth

here is calculated solely using standard OpenQASM gates (see Section 2.2). That is to say, the OpenQASM code is decomposed to standard gates before counting the accurate time evolution steps required. As a concrete example, an X gate is a standard gate and hence requires 1 time-step, whereas a CRZ gate requires 4 standard gates to complete, thereby requiring 4 timesteps.

Circuit depth can be computed by decomposing OpenQASM code into a $n_q \times t$ matrix Q , where Q_{q_i, t_j} is the time-evolution steps to complete the gate on qubit i at time j . The sum of the maximum time in each column is then equal to the minimum time required for a quantum application. This is described in Equation (2):

$$\text{Circuit Depth} = \sum_{j=1}^t \max_{0 \leq i < \text{width}} (Q_{q_i, t_j}) \quad (2)$$

4.3 Gate Density

Gate density, or operation density, describes the occupancy of gate slots along the time-evolution steps of a quantum circuit. As certain qubits might need to wait for other qubits in the time evolution (i.e., gate dependency), they remain idle by executing the ID gate (see Table 2). Consequently, if a gate slot is empty due to dependency, it implies a lower occupancy for the quantum hardware. This is similar to a classical processor, where data dependency introduces pipeline bubbles and reduced occupancy. We propose Gate Density to measure the likely occupancy of a circuit when mapping to a quantum hardware, as formulated in Equation (3). Note, the gate density is also calculated after decomposing down to the OpenQASM standard gates.

$$\text{Gate Density} = \frac{G_{1\text{-qubit}} + 2 \times G_{2\text{-qubit}}}{\text{Circuit Depth} \times \text{Circuit Width}} \quad (3)$$

where $G_{1\text{-qubit}}$ refers to the number of 1-qubit standard gates and $G_{2\text{-qubit}}$ refers to the number of 2-qubit standard gates (see Table 2).

4.4 Retention Lifespan

Retention Lifespan describes the maximum lifespan of a qubit within a system, and is motivated by the T1 and T2 coherence time of a quantum device (see Section 2.1). A longer lifespan of a quantum system implies more decay to the ground state (T1) and state-transition due to environment noise (T2), thus is more susceptible to information loss. Therefore, we propose taking the qubit with the longest lifespan to determine the system's retention lifespan. Using this metric, one can estimate if a particular circuit can be executed in a NISQ device with high fidelity, given its T1/T2 coherence time. Note, all IBM-Q machines offer T1/T2 coherence time as status indicators for the hardware. As circuit depth can grow substantially, we introduce the log operator to shrink the scale. If D_i is used to describe the lifespan or depth of the qubit i , Retention Lifespan, which measures the circuit size and sensitivity to quantum system error, is defined in Equation (4):

$$\text{Retention Lifespan} = \max_{0 \leq i < \text{width}} (\log(D_i)) \quad (4)$$

4.5 Measurement Density

Measurement density assesses the importance of measurements in a circuit. A higher measurement count implies the fact that each measurement might be of relatively less importance (e.g., periodic measurement in QEC, or measurement over ancilla qubits), whereas for application with less measurements, the measurement may be of utmost importance. The importance also increases when a measurement accounts for a wider and/or deeper circuit. A good example is the SWAP test (see Figure 7), where the circuit can be very large but only one measurement is taken to report the

similarity. Consequently, this measurement is extremely important to the application. Measurement Density is defined in Equation (5), where $N_{\text{measurement}}$ is the number of measurements in an application. Since the circuit depth/width can be large and the importance of measurement decays when circuit depth/width keeps on increasing, we add a log to shrink the scale.

$$\text{Measurement Density} = \frac{\log(\text{Circuit Depth} \times \text{Circuit Width})}{N_{\text{measurement}}} \quad (5)$$

4.6 Entanglement Variance

Entanglement Variance measures the balance of entanglement across the qubits of a circuit. Circuits with a higher Entanglement Variance indicate that certain qubits are more connected than other qubits (i.e., using more 2-qubit gates such as CX than others). This metric implies that when the circuit is mapped to a NISQ device: (i) fewer SWAP gates are needed if those hotspot qubits are mapped to the central vertices in the NISQ device topology, such as Qubit-1 in *ibmq_belem* and Qubit-2 in *ibmq_yorkton* in Figure 2). A higher entanglement variance implies a higher potential benefit from a good logic-physical qubit-mapping through quantum transpilation. If the entanglement variance is zero, little benefit should be expected from a better transpilation strategy; (ii) Given 2-qubit gate is one of the major sources introducing error, a higher Entanglement Variance implies uneven error introduction among qubits. We define Entanglement Variance in Equation (6), where $G_{q_i}(2\text{-qubits})$ is the number of 2-qubit gates operating on qubit- i , and $\overline{G_q}$ is the average number of 2-qubit gates operating over each qubit. The inclusion of plus 1 is to eliminate log errors with 0 variance.

$$\text{Entanglement Variance} = \frac{\log(\sum_{i=0}^{\text{width}} (G_{q_i}(2\text{-qubits}) - \overline{G_q}(2\text{-qubits}))^2 + 1)}{\text{Circuit Width}} \quad (6)$$

5 EVALUATION

We first evaluate QASMBench circuits using the proposed metrics and then benchmark a subset of the circuits on real NISQ devices.

5.1 Circuit Metrics

We evaluate QASMBench applications described in Section 3 using the circuit metrics proposed in Section 4.

5.1.1 Circuit Width. Figure 8 illustrates the width of the circuits in QASMBench. Since we partition QASMBench with respect to the number of qubits (1-5 for small-scale, 6-15 for medium-scale, and 15-127 for large-scale), this can be seen in Figure 8.

5.1.2 Circuit Depth. Figure 9 shows the depth of the benchmark circuits from QASMBench. Apparently, some circuits (e.g., *DNN*, *VQE*, *QFT*) are significantly deeper than other circuits. This figure shows that QASMBench covers a wide range of circuit depth from 11 in the *Linearsolver_n-3* to 14,281 in the *QFT_n-85*, offering a comprehensive circuit database for evaluating NISQ capacity and stability.

5.1.3 Gate Density. The plot of Gate Density for QASMBench is visualized in Figure 10. The general observation is that, with larger scale circuits, the gate density tends to be more sparse, implying reduced execution occupancy or efficiency. On the contrary, smaller circuits tend to be denser and more carefully designed, so that qubits are less likely to wait for other qubits before evolving onto the next time step. Nevertheless, QASMBench includes densely packed large and medium circuits with *Ising_n-26* having a Gate Density factor of 0.8462 for large scale circuits, and *BB84_n-8* with a Gate Density of 0.675 in the medium scale category. Note, a low gate density

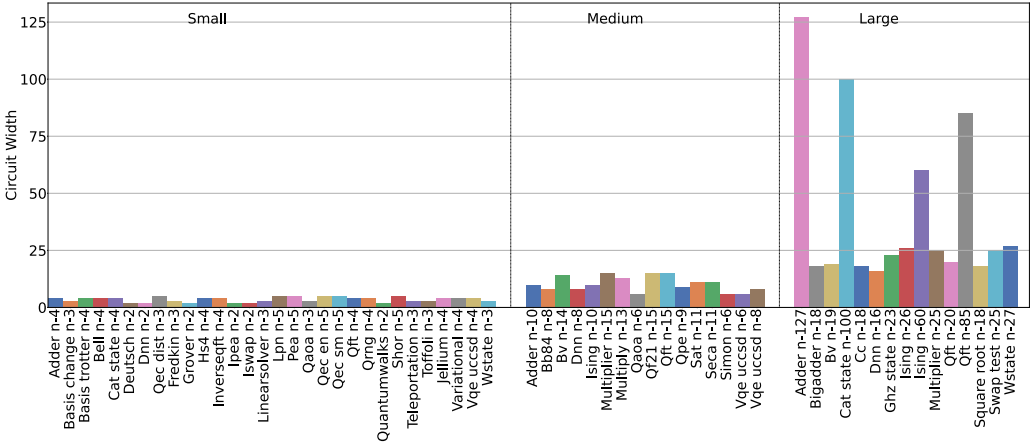


Fig. 8. Circuit width of QASMBench circuits with respect to the small, medium, and large categories.

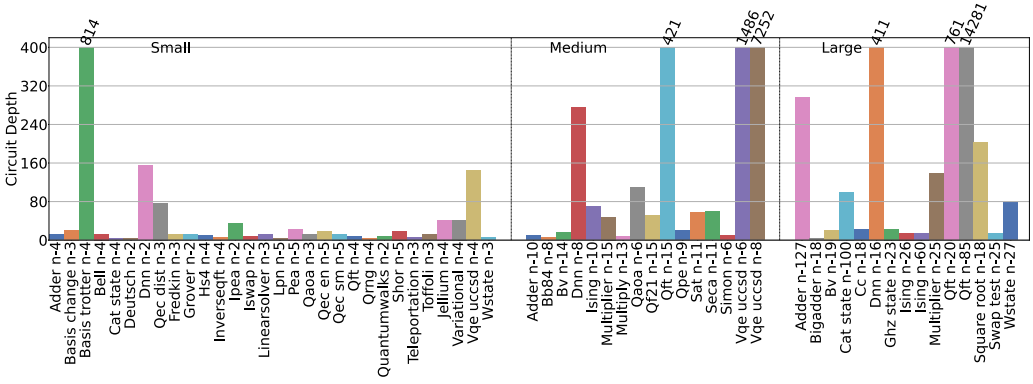


Fig. 9. Circuit depth of QASMBench circuits.

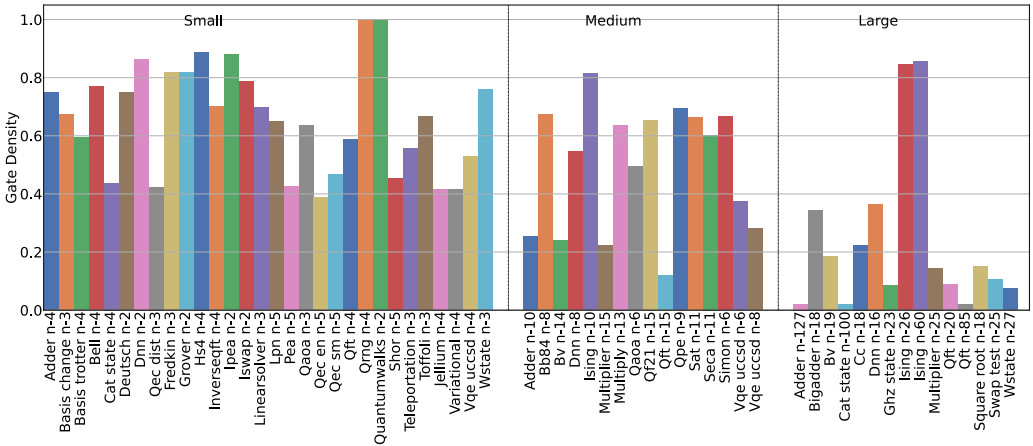


Fig. 10. Gate density of QASMBench circuits.

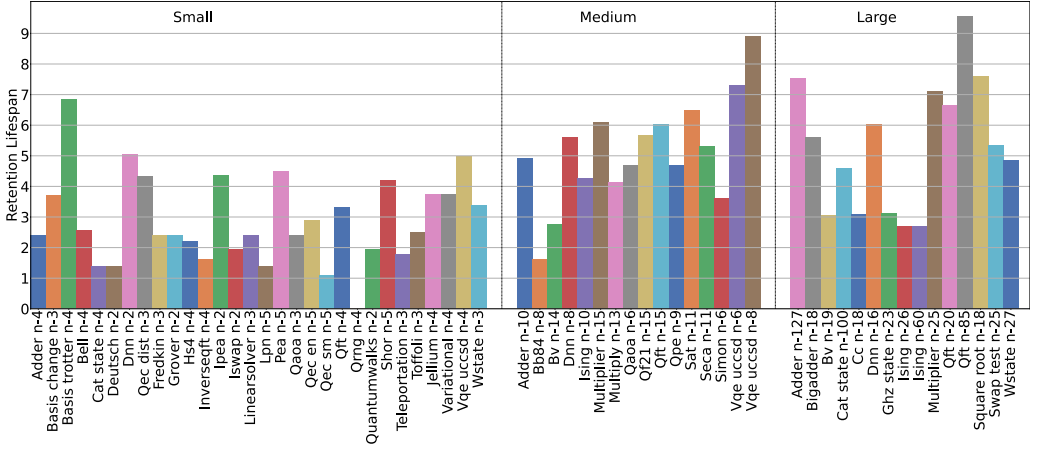


Fig. 11. Retention lifespan of QASMBench circuits.

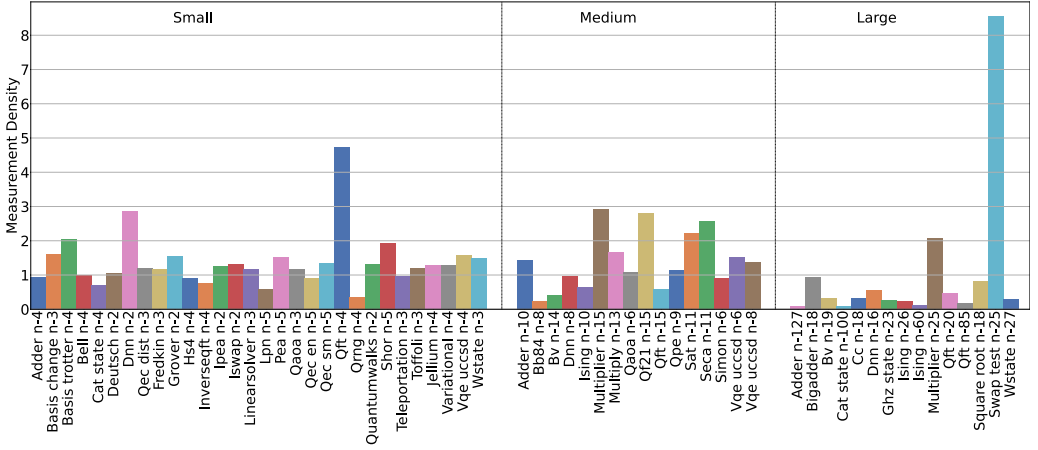


Fig. 12. Measurement density of QASMBench circuits.

may also imply the amount of internal operation dependency and the potential gain with a better circuit reform.

5.1.4 Retention Lifespan. Retention Lifespan, visualized in Figure 11, illustrates the highly varying maximum qubit lifespan within the benchmark circuits of QASMBench. Short circuits such as *Deutsch_n-2* are required to preserve a quantum state for a relatively short period of time with a retention lifespan of 1.3863, compared to the *Square-Root_n-18* circuit with a retention life span of 5.6168. Note, retention lifespan is related to circuit depth, but taking dependency into consideration and is specific to a certain qubit rather than the overall system.

5.1.5 Measurement Density. The measurement density for the QASMBench circuits is illustrated in Figure 12, which measures the importance of measurement operations in a circuit. As can be seen, *SWAP-test*, *multiplier*, *DNN*, and *QFT* show significantly higher measurement density than the other circuits. We have already discussed Swap-Test in Section 3.5. *QFT* has a very deep circuit which explains why the only measurement at the end is of more importance. For

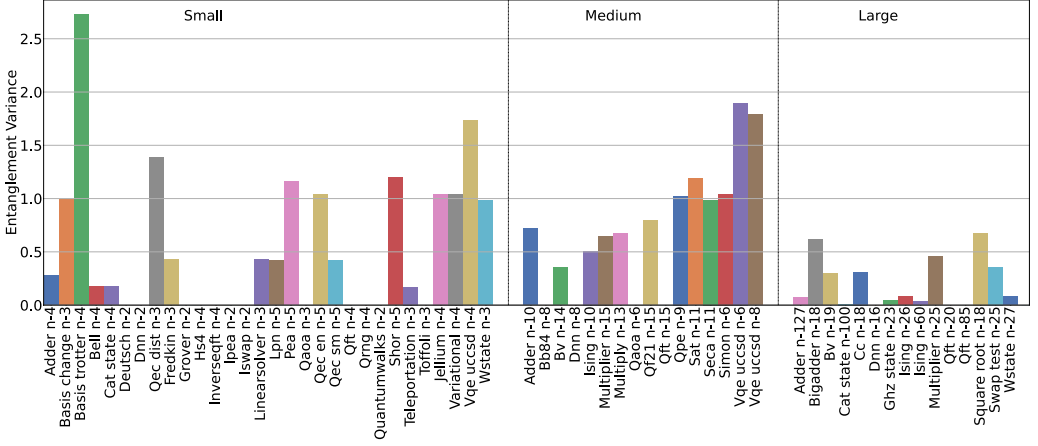


Fig. 13. Entanglement variance of QASMBench circuits.

multiplier, only qubits storing the product results are measured at the end. DNN essentially embeds the SWAP-test as a component for the estimation of the cost function.

5.1.6 Entanglement Variance. Figure 13 shows the entanglement variance for the QASMBench circuits. The figure demonstrates QASMBench’s diverse entanglement arrangements, with densely entangled circuits such as DNN n-2 and Grover n-2, where all entanglement operations entangle the only two qubits available. Therefore, the entanglement is evenly distributed across the circuit. However, compared to Basis trotter n-4, the primary bulk of entanglement is between qubits 1 and 2, whereas qubits 0 and 4 interact substantially less with other qubits. Consequently, when mapping *Basis-trotter* to a NISQ device like *ibmq_belem*, it involves much fewer SWAP gates if qubit-1 and 2 are mapped to Q1 and Q3 in Figure 2 than the reverse.

5.2 Evaluating QASMBench on Real NISQ Devices

We first describe the metric we used for the evaluation. Then, we evaluate applications from the small category of QASMBench on 12 IBM-Q machines. Finally, we compare among devices from three different NISQ hardware vendors.

5.2.1 Evaluation Metric. We use **fidelity** to assess the quality of executing a particular quantum algorithm in a specific quantum device. Fidelity here is defined as the distance between the actual quantum state of a NISQ device with the ideal quantum state obtained from a classical simulation (e.g., [77, 78]). Since NISQ devices are susceptible to noise, the actual quantum state is essentially a mixed state, described by a density matrix. However, quantum observables are not directly measurable on the classical side. Therefore, we reconstruct this mixed state density matrix through state tomography.

Equation (7) shows the definition of fidelity we used in the evaluation. Its average value at runtime reflects a quantum machine’s ability to induce a particular circuit.

$$F(\rho, \sigma) = \left(\text{Tr} \sqrt{\sqrt{\rho} \sigma \sqrt{\rho}} \right)^2 \quad (7)$$

where ρ is the density matrix representing the pure state $|\psi_\rho\rangle$ from a classical simulation. σ is the density matrix representing the mixed state $|\psi_\sigma\rangle$ from the noisy execution in the NISQ device.

Since $|\psi_\rho\rangle$ is a pure state, Equation (7) can be further simplified as:

$$F(\rho, \sigma) = \langle \psi_\rho | \sigma | \psi_\rho \rangle \quad (8)$$

We use Equation (8) to measure the fidelity, which should be the most precise fidelity metric, to the best of our knowledge. However, density matrix tomography is extremely computationally expensive, which requires 3^n times' circuit evaluations where n is the number of qubits. Therefore, for a 10-qubit circuit, it demands 59,049 circuit evaluations (without even counting the shots per evaluation). Additionally, these evaluations should be performed consecutively in a short time window and at least within the same calibration period. Otherwise, the state to be reconstructed is not consistent any more. Consequently, such an approach is only feasible for circuits with a small number of qubits. Nevertheless, active research is ongoing in this area to accelerate and make state tomography more scalable through technologies such as neural networks [108].

5.2.2 IBM-Q Evaluation. Due to the excessive cost of density matrix tomography, we evaluate 24 circuits from the small category (see Table 3) on 12 IBM-Q machines: Washington (127-qubit, QV64 which means quantum volume is 64), Guadalupe (16-qubit, QV32), Toronto (27-qubit, QV32), Brooklyn (65-qubit, QV32), Perth (7-qubit, QV32), Cairo (27-qubit, QV64), Jakarta (7-qubit, QV16), Montreal (27-qubit, QV128), Mumbai (27-qubit, QV128), Auckland (27-qubit, QV64), Lagos (7-qubit, QV32), and Hanoi (7-qubit, QV32), through PNNL's IBM-Q Hub. We perform density matrix state tomography to calculate the fidelity using Equation (8) for each circuit, with respect to the pure state from the state-vector simulator of Qiskit. This effort comprises 25,000 circuit evaluations in total. We use 8,192 shots per evaluation. The Qiskit version is 0.33.0. We use the default optimization level for the Qiskit transpiler.

Figure 14 illustrates the results. We observe that: (i) Compared among different benchmark circuits, the fidelity is generally correlated with circuit depth and width. For example, the circuit depths of `basis_trotter`, `qec_dist3`, and `vqe_uccsd` are much larger than the other circuits, with `qec_en` and `qft` stay in the middle. For `dnn n-2`, although the circuit is relatively deep (268 gates), due to less qubit, the fidelity level is better than `adder n-2` with merely 23 gates. (ii) Compared among different machines, the fidelity appears to be weakly correlated with the qubit number of the machine and its quantum volume. Particularly, for `qnn n-2`, despite mapping to the best performing qubits of IBM-Q Washington, the fidelity level is still lower than the 7-qubit IBM-Q Hanoi with a less QV. (iii) In general, the impact from the noise, e.g., the decay noise, is significant. With merely 36 gates in `qft n-4`, the fidelity for most machines drops below 0.6 except Hanoi.

5.2.3 Evaluations of NISQ Devices. We further evaluate four circuits (i.e., Deutsch, DNN, Grover, and iSWAP) on different NISQ devices. We use the 5-qubit IonQ trapped-ion QPU through Microsoft Azure Quantum [92], and the 80-qubit Rigetti Aspen M-1 superconducting machine through Oak Ridge Leadership Computing Facility. We compare the fidelity with IBM-Q Brooklyn (65-qubit, QV32), Lima (5-qubit, QV8), Hanoi (27-qubit, QV64), and Washington (127-qubit, QV64). The results are shown in Figure 15. The modest evaluation is due to resource limitations.

Through this evaluation, we can observe that IonQ's trapped-ion system generally obtains higher fidelity than the other devices, for relatively shallow circuits with smaller retention lifespan. However, for a deeper circuit, the performance drops significantly compared to the other superconducting devices. This can be attributed to the lower single-gate fidelity level for the trapped ion processor (i.e., 99.35%), compared with the other superconducting devices (e.g., 99.90% for IBM-Q Washington). Despite the higher T1 time ($\geq 10^7 \mu s$) of IonQ's QPU can preserve quantum information better (with respect to $99.28 \mu s$ for IBM-Q Washington), the exponential compounding imperfect gate fidelity leads to an observable reduction in the overall performance.

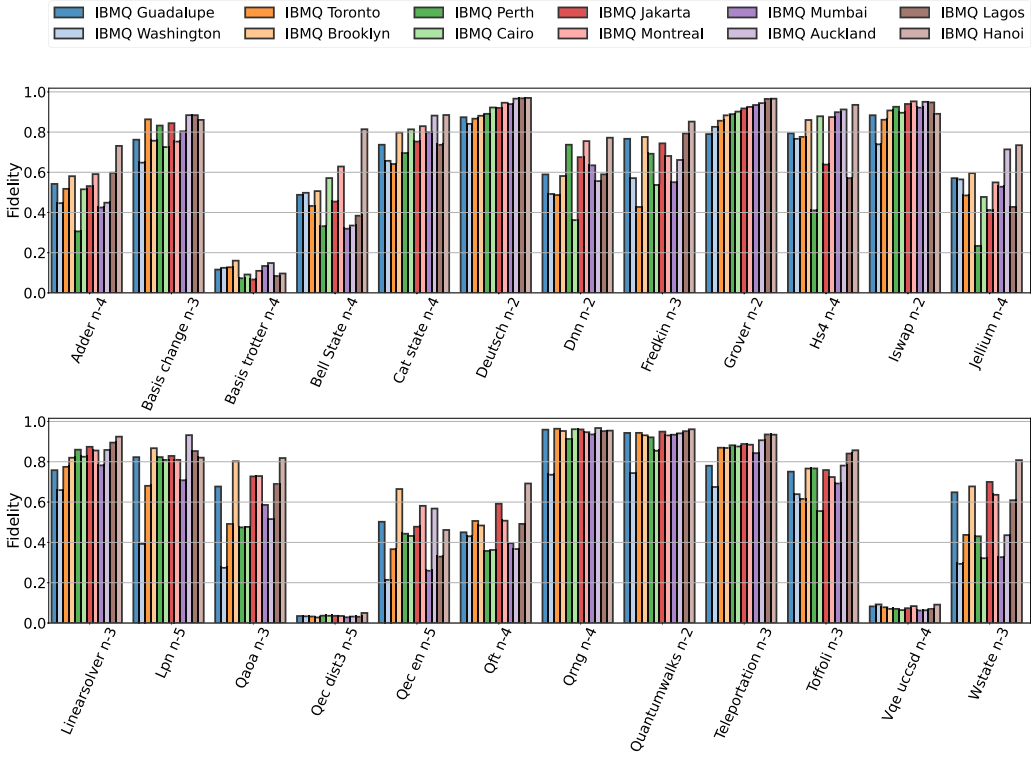


Fig. 14. Evaluations of 24 circuits from QASMBench on 12 IBM-Q machines via density matrix tomography.

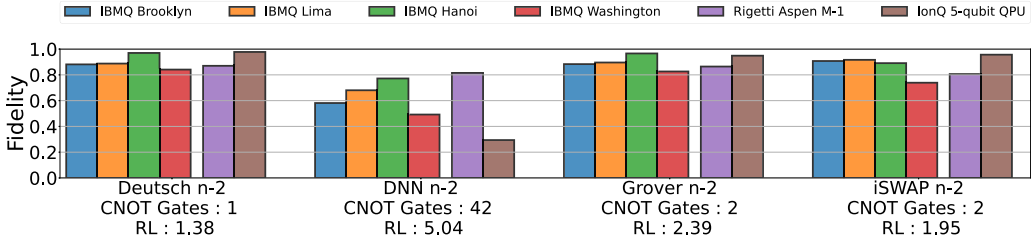


Fig. 15. Fidelity evaluation on 4 IBM-Q machines, IonQ's 5-qubit QPU and Rigetti's 80-qubit Aspen M-1 machine. RL refers to the retention lifespan of the circuit (see Equation (4)).

6 RELATED WORK

To the best of our knowledge, QASMBench is the first benchmark suite aiming at evaluating NISQ devices using quantum applications from a broad range of domains. Existing works have already proposed to adopt randomly generated benchmarks for evaluating the error rates of different gate operations regarding a particular NISQ device [83, 84, 106]. Recently, Patel et al. [103] evaluated several IBM-Q machines using seven benchmarks and drew interesting observations regarding the error and execution time. Since the work mainly focused on extracting and validating new insights, only limited applications were included and no new metrics had been proposed.

Regarding QC metrics, Cross et al. defined the quantum volume protocol which quantified the largest random circuit of equal width and depth that a NISQ device can successfully implement [28],

taking gate and measurement error into consideration. Quantum Linpack, proposed by Dong and Lin [36] recently, attempted to develop a **random circuit block-encoded matrix (RACBEM)** for measuring the QC capability in resolving a random system of linear equations. Benedetti et al. [10] proposed the qBAS score metric for benchmarking the shallow quantum-classical hybrid systems using synthetic data set. These metrics, despite being comprehensive, are not quite intuitive for understanding. Besides, these metrics all involve certain randomness due to synthetically generated inputs or circuits, which do not necessarily share the properties of real quantum workloads. Given the randomness, the validation requires offline simulation in a classic computer, ultimately limiting the scalability as the simulation cost in a classical computer scales exponentially with increased number of qubits.

Regarding the general evaluation of NISQ devices, in addition to [103], LaRose [73] compared the four widely used QC programming environments: Forest/pyQuil, Qiskit, Project, and QDK/Q# in terms of library support, hardware, compiler, and simulator performance. McCaskey et al. [86] proposed XACC, a programming model that defines a low-level intermediate representation and compiler front-end to provide a unified abstraction for evaluating different quantum hardware and programming environments. Finally, Abhijith et al. [3] developed a great hands-on tutorial in explaining the principles of QC using 20 different quantum algorithms, using the 5-qubit IBM-Q machine as the validation platform. However, this work is mainly for education purposes using the small-scale circuits (5 qubits) rather than serving as a QC benchmark suite. SupermarQ [128] comprises a set of circuits in attempts to benchmark the current term quantum processors. Lubinski et al. evaluated NISQ processors comprehensively according to volumetric benchmarking, and provided evaluations of quantum volume on a diverse set of NISQ superconducting and trapped ion quantum processors [82].

7 CONCLUSION

In this paper, we propose a light-weighted, low-level and easy-to-use benchmark suite called QASMBench based on the OpenQASM quantum assembly language. It collects commonly seen quantum algorithms and routines from a variety of domains with distinct properties, serving the need for convenient quantum computing characterization and evaluation purposes for the computer system and architecture communities. Additionally, we propose four novel circuit metrics including gate density, retention lifespan, measurement density, and entanglement variance for assessing the execution efficiency, susceptibility to NISQ error, and potential gain from low-level optimizations. We also evaluate the fidelity of inducing QASMBench circuits on IBM-Q, IonQ and Rigetti NISQ devices through density matrix state tomography and derive new observations. QASMBench, together with the metrics and evaluations, provides a unique tool for evaluating and characterizing the properties of emerging NISQ devices, estimating the potential of optimization from quantum transpilers, and benchmarking the performance of classical quantum simulators.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their helpful comments.

REFERENCES

- [1] [n.d.]. Quantum programming language converter.
- [2] [n.d.]. Quantum programming language converter.
- [3] J. Abhijith, Adetokunbo Adedoyin, John Ambrosiano, Petr Anisimov, Andreas Bärttschi, William Casper, Gopinath Chennupati, Carleton Coffrin, Hristo Djidjev, David Gunter, et al. 2018. Quantum algorithm implementations for beginners. *arXiv e-prints* (2018), arXiv-1804.
- [4] Dorit Aharonov, Andris Ambainis, Julia Kempe, and Umesh Vazirani. 2001. Quantum walks on graphs. In *Proceedings of the Thirty-third Annual ACM Symposium on Theory of Computing*. 50–59.

- [5] Thorsten Altenkirch and Jonathan Grattage. 2005. A functional quantum programming language. In *20th Annual IEEE Symposium on Logic in Computer Science (LICS'05)*. IEEE, 249–258.
- [6] ANAKIN. 2019. <https://github.com/AgentANAKIN/>.
- [7] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574, 7779 (2019), 505–510.
- [8] Alán Aspuru-Guzik and Philip Walther. 2012. Photonic quantum simulators. *Nature Physics* 8, 4 (2012), 285–291.
- [9] Carlos Barrón-Romero. 2015. Classical and quantum algorithms for the Boolean satisfiability problem. *arXiv preprint arXiv:1510.02682* (2015).
- [10] Marcello Benedetti, Delfina Garcia-Pintos, Oscar Perdomo, Vicente Leyton-Ortega, Yunseong Nam, and Alejandro Perdomo-Ortiz. 2019. A generative modeling approach for benchmarking and training shallow quantum circuits. *npj Quantum Information* 5, 1 (2019), 1–9.
- [11] Paul Benioff. 1980. The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. *Journal of Statistical Physics* 22, 5 (1980), 563–591.
- [12] Charles H. Bennett and Gilles Brassard. 2020. Quantum cryptography: Public key distribution and coin tossing. *arXiv preprint arXiv:2003.06557* (2020).
- [13] Ethan Bernstein and Umesh Vazirani. 1997. Quantum complexity theory. *SIAM Journal on Computing* 26, 5 (1997), 1411–1473.
- [14] Dominic W. Berry, Graeme Ahokas, Richard Cleve, and Barry C. Sanders. 2007. Efficient quantum algorithms for simulating sparse Hamiltonians. *Communications in Mathematical Physics* 270, 2 (2007), 359–371.
- [15] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. 2017. Quantum machine learning. *Nature* 549, 7671 (2017), 195–202.
- [16] BramDo. 2017. Quantum Examples QASM. https://github.com/BramDo/quantum_examples_qasm.
- [17] Cyril Branciard, Nicolas Gisin, Barbara Kraus, and Valerio Scarani. 2005. Security of two quantum cryptography protocols using the same four qubit states. *Physical Review A* 72, 3 (2005), 032301.
- [18] Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J. Martinez, Jae Hyeon Yoo, Sergei V. Isakov, Philip Massey, Murphy Yuezhen Niu, Ramin Halavati, Evan Peters, et al. 2020. TensorFlow quantum: A software framework for quantum machine learning. *arXiv preprint arXiv:2003.02989* (2020).
- [19] Bikas K. Chakrabarti, Amit Dutta, and Parongama Sen. 2008. *Quantum Ising Phases and Transitions in Transverse Ising Models*. Vol. 41. Springer Science & Business Media.
- [20] Bo Chen, Zhigang Ye, Chen Chen, and Jianhui Wang. 2018. Toward a MILP modeling framework for distribution system restoration. *IEEE Transactions on Power Systems* 34, 3 (2018), 1749–1760.
- [21] Frederic T. Chong. 2018. Quantum computing is getting real: Architecture, PL, and OS roles in closing the gap between quantum algorithms and machines. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*. 285–285.
- [22] Lukasz Cincio, Yiğit Subaşı, Andrew T. Sornborger, and Patrick J. Coles. 2018. Learning the quantum algorithm for state overlap. *New Journal of Physics* 20, 11 (2018), 113022.
- [23] Juan I. Cirac and Peter Zoller. 1995. Quantum computations with cold trapped ions. *Physical Review Letters* 74, 20 (1995), 4091.
- [24] B. David Clader, Bryan C. Jacobs, and Chad R. Sprouse. 2013. Preconditioned quantum linear system algorithm. *Physical Review Letters* 110, 25 (2013), 250504.
- [25] John Clarke and Frank K. Wilhelm. 2008. Superconducting quantum bits. *Nature* 453, 7198 (2008), 1031–1042.
- [26] Don Coppersmith. 2002. An approximate Fourier transform useful in quantum factoring. *arXiv preprint quant-ph/0201067* (2002).
- [27] Antonio D. Córcoles, Abhinav Kandala, Ali Javadi-Abhari, Douglas T. McClure, Andrew W. Cross, Kristan Temme, Paul D. Nation, Matthias Steffen, and J. M. Gambetta. 2019. Challenges and opportunities of near-term quantum computing systems. *arXiv preprint arXiv:1910.02894* (2019).
- [28] Andrew W. Cross, Lev S. Bishop, Sarah Sheldon, Paul D. Nation, and Jay M. Gambetta. 2019. Validating quantum computers using randomized model circuits. *Physical Review A* 100, 3 (2019), 032328.
- [29] Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. 2017. Open quantum assembly language. *arXiv preprint arXiv:1707.03429* (2017). Repo: <https://github.com/Qiskit/openqasm>.
- [30] Andrew W. Cross, Graeme Smith, and John A. Smolin. 2015. Quantum learning robust against noise. *Physical Review A* 92, 1 (2015), 012327.
- [31] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. 2004. A new quantum ripple-carry addition circuit. *arXiv preprint quant-ph/0410184* (2004).
- [32] Poulami Das, Swamit S. Tannu, Prashant J. Nair, and Moinuddin Qureshi. 2019. A case for multi-programming quantum computers. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 291–303.

- [33] David Deutsch and Richard Jozsa. 1992. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences* 439, 1907 (1992), 553–558.
- [34] David P. DiVincenzo. 2000. The physical implementation of quantum computation. *Fortschritte der Physik: Progress of Physics* 48, 9–11 (2000), 771–783.
- [35] Miroslav Dobšiček, Göran Johansson, Vitaly Shumeiko, and Göran Wendin. 2007. Arbitrary accuracy iterative quantum phase estimation algorithm using a single ancillary qubit: A two-qubit benchmark. *Physical Review A* 76, 3 (2007), 030306.
- [36] Yulong Dong and Lin Lin. 2020. Random circuit block-encoded matrix and a proposal of quantum LINPACK benchmark. *arXiv preprint arXiv:2006.04010* (2020).
- [37] Eugene F. Dumitrescu, Alex J. McCaskey, Gaute Hagen, Gustav R. Jansen, Titus D. Morris, T. Papenbrock, Raphael C. Pooser, David Jarvis Dean, and Pavel Lougovski. 2018. Cloud quantum computing of an atomic nucleus. *Physical Review Letters* 120, 21 (2018), 210501.
- [38] Wolfgang Dür, Guifre Vidal, and J. Ignacio Cirac. 2000. Three qubits can be entangled in two inequivalent ways. *Physical Review A* 62, 6 (2000), 062314.
- [39] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).
- [40] Edward Farhi and Sam Gutmann. 1998. Quantum computation and decision trees. *Physical Review A* 58, 2 (1998), 915.
- [41] Serguei Fedortchenko. 2016. A quantum teleportation experiment for undergraduate students. *arXiv preprint arXiv:1607.02398* (2016).
- [42] Richard P. Feynman. 1999. Simulating physics with computers. *Int. J. Theor. Phys.* 21, 6/7 (1999).
- [43] Mark Fingerhuth. 2020. Open-Source Quantum Software Projects. <https://github.com/qosf/awesome-quantum-software>.
- [44] Michael Fleischhauer and Mikhail D. Lukin. 2002. Quantum memory for photons: Dark-state polaritons. *Physical Review A* 65, 2 (2002), 022314.
- [45] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. 2012. Surface codes: Towards practical large-scale quantum computation. *Physical Review A* 86, 3 (2012), 032324.
- [46] Xiang Fu, Michiel Adriaan Rol, Cornelis Christiaan Bultink, J. Van Someren, Nader Khammassi, Imran Ashraf, RFL Vermeulen, J. C. De Sterke, W. J. Vlothuizen, R. N. Schouten, et al. 2017. An experimental microarchitecture for a superconducting quantum processor. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 813–825.
- [47] John King Gamble, Mark Friesen, Dong Zhou, Robert Joynt, and S. N. Coppersmith. 2010. Two-particle quantum walks applied to the graph isomorphism problem. *Physical Review A* 81, 5 (2010), 052313.
- [48] Iulia M. Georgescu, Sahel Ashhab, and Franco Nori. 2014. Quantum simulation. *Reviews of Modern Physics* 86, 1 (2014), 153.
- [49] Nicolas Gisin, Grégoire Ribordy, Wolfgang Tittel, and Hugo Zbinden. 2002. Quantum cryptography. *Reviews of Modern Physics* 74, 1 (2002), 145.
- [50] Constantín González. 2021. Cloud based QC with Amazon Braket. *Digitale Welt* 5, 2 (2021), 14–17.
- [51] Google. [n.d.]. <https://github.com/quantumlib/Cirq>.
- [52] Daniel M. Greenberger, Michael A. Horne, and Anton Zeilinger. 1989. Going beyond Bell's theorem. In *Bell's Theorem, Quantum Theory and Conceptions of the Universe*. Springer, 69–72.
- [53] Harper R. Grimsley, Daniel Claudino, Sophia E. Economou, Edwin Barnes, and Nicholas J. Mayhall. 2019. Is the trotterized UCCSD ansatz chemically well-defined? *Journal of Chemical Theory and Computation* (2019).
- [54] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-eighth Annual ACM Symposium on Theory of Computing*. 212–219.
- [55] Lov K. Grover. 1998. Quantum computers can search rapidly by using almost any transformation. *Physical Review Letters* 80, 19 (1998), 4329.
- [56] Thomas Häner, Damian S. Steiger, Torsten Hoefler, and Matthias Troyer. 2021. Distributed quantum computing with QMPI. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–13.
- [57] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. 2009. Quantum algorithm for linear systems of equations. *Physical Review Letters* 103, 15 (2009), 150502.
- [58] Ni-Ni Huang, Wei-Hao Huang, and Che-Ming Li. 2020. Identification of networking quantum teleportation on 14-qubit IBM universal quantum computer. *Scientific Reports* 10, 1 (2020), 1–12.
- [59] IBM. [n.d.]. IBM Quantum Experience. <https://quantum-computing.ibm.com/>.
- [60] IBM. [n.d.]. Qiskit: Elements for building a quantum future. <https://github.com/Qiskit/qiskit>.
- [61] IonQ. [n.d.]. Best Practices for Using IonQ Hardware. <https://ionq.com/best-practices>.

- [62] Joseph T. Iosue. [n.d.]. QAOAPython: The Quantum Approximate Optimization Algorithm implemented on Cirq, ProjectQ, and Qiskit. <https://github.com/jtiosue/QAOAPython>.
- [63] Kazuo Iwama, Harumichi Nishimura, Rudy Raymond, and Junichi Teruyama. 2010. Quantum counterfeit coin problems. In *International Symposium on Algorithms and Computation*. Springer, 85–96.
- [64] Kazuo Iwama, Harumichi Nishimura, Rudy Raymond, and Junichi Teruyama. 2012. Quantum counterfeit coin problems. *Theoretical Computer Science* 456 (2012), 51–64.
- [65] Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. 2014. ScaffCC: A framework for compilation and analysis of quantum computing programs. In *Proceedings of the 11th ACM Conference on Computing Frontiers*. 1–10. Repo: <https://github.com/epiqc/ScaffCC>.
- [66] Tyson Jones, Suguru Endo, Sam McArdle, Xiao Yuan, and Simon C. Benjamin. 2019. Variational quantum algorithms for discovering Hamiltonian spectra. *Physical Review A* 99, 6 (2019), 062304.
- [67] Stephen Jordan. [n.d.]. Quantum Algorithm Zoo. <https://quantumalgorithmzoo.org/>.
- [68] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M. Chow, and Jay M. Gambetta. 2017. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature* 549, 7671 (2017), 242–246.
- [69] Min-Sung Kang, Jino Heo, Seong-Gon Choi, Sung Moon, and Sang-Wook Han. 2019. Implementation of SWAP test for two unknown states in photons via cross-Kerr nonlinearities under decoherence effect. *Scientific Reports* 9, 1 (2019), 1–14.
- [70] Jordanis Kerenidis and Anupam Prakash. 2016. Quantum recommendation systems. *arXiv preprint arXiv:1603.08675* (2016).
- [71] H. Jeff Kimble. 2008. The quantum internet. *Nature* 453, 7198 (2008), 1023–1030.
- [72] Henning Labuhn, Daniel Barredo, Sylvain Ravets, Sylvain De Léséleuc, Tommaso Macrì, Thierry Lahaye, and Antoine Browaeys. 2016. Tunable two-dimensional arrays of single Rydberg atoms for realizing quantum Ising models. *Nature* 534, 7609 (2016), 667–670.
- [73] Ryan LaRose. 2019. Overview and comparison of gate level quantum software platforms. *Quantum* 3 (2019), 130.
- [74] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. IEEE, 75–86.
- [75] Dietrich Leibfried, Rainer Blatt, Christopher Monroe, and David Wineland. 2003. Quantum dynamics of single trapped ions. *Reviews of Modern Physics* 75, 1 (2003), 281.
- [76] Dietrich Leibfried, Emanuel Knill, Signe Seidelin, Joe Britton, R. Brad Blakestad, John Chiaverini, David B. Hume, Wayne M. Itano, John D. Jost, Christopher Langer, et al. 2005. Creation of a six-atom ‘Schrödinger cat’ state. *Nature* 438, 7068 (2005), 639–642.
- [77] Ang Li, Bo Fang, Christopher Granade, Guen Prawiroatmodjo, Bettina Heim, Martin Roetteler, and Sriram Krishnamoorthy. 2021. SV-Sim: Scalable PGAS-based state vector simulation of quantum circuits. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
- [78] Ang Li, Omer Subasi, Xiu Yang, and Sriram Krishnamoorthy. 2020. Density matrix quantum circuit simulation via the BSP machine on modern GPU clusters. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [79] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the qubit mapping problem for NISQ-era quantum devices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1001–1014.
- [80] Gushu Li, Yufei Ding, and Yuan Xie. 2020. Towards efficient superconducting quantum processor architecture design. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1031–1045.
- [81] Seth Lloyd. 1996. Universal quantum simulators. *Science* (1996), 1073–1078.
- [82] Thomas Lubinski, Sonika Johri, Paul Varosy, Jeremiah Coleman, Luning Zhao, Jason Necaie, Charles H. Baldwin, Karl Mayer, and Timothy Proctor. 2021. Application-oriented performance benchmarks for quantum computing. *arXiv preprint arXiv:2110.03137* (2021).
- [83] Easwar Magesan, Jay M. Gambetta, and Joseph Emerson. 2011. Scalable and robust randomized benchmarking of quantum processes. *Physical Review Letters* 106, 18 (2011), 180504.
- [84] Easwar Magesan, Jay M. Gambetta, and Joseph Emerson. 2012. Characterizing quantum gates via randomized benchmarking. *Physical Review A* 85, 4 (2012), 042311.
- [85] R. Maurand, X. Jehl, D. Kotekar-Patil, A. Corna, H. Bohuslavskyi, R. Laviéville, L. Hutin, S. Barraud, M. Vinet, M. Sanquer, et al. 2016. A CMOS silicon spin qubit. *Nature Communications* 7, 1 (2016), 1–6.
- [86] Alexander J. McCaskey, Eugene F. Dumitrescu, Dmitry Liakh, Mengsu Chen, Wu-chun Feng, and Travis S. Humble. 2018. A language and hardware independent approach to quantum-classical computing. *SoftwareX* 7 (2018), 245–254.

- [87] Jarrod McClean, Nicholas Rubin, Kevin Sung, Ian David Kivlichan, Xavier Bonet-Monroig, Yudong Cao, Chengyu Dai, Eric Schuyler Fried, Craig Gidney, Brendan Gimby, et al. 2020. OpenFermion: The electronic structure package for quantum computers. *Quantum Science and Technology* (2020). Repo: <https://github.com/quantumlib/OpenFermion-Cirq>.
- [88] Jarrod R. McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. 2016. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics* 18, 2 (2016), 023023.
- [89] Raffaele Miceli. [n.d.]. Quantum_Walks: Qiskit code to simulate quantum walks on graphs with up to 4 nodes. https://github.com/raffmiceli/Quantum_Walks.
- [90] Kristel Michielsen, Madita Nocon, Dennis Willsch, Fengping Jin, Thomas Lippert, and Hans De Raedt. 2017. Benchmarking gate-based quantum computers. *Computer Physics Communications* 220 (2017), 44–55.
- [91] Microsoft. [n.d.]. Azure Honeywell provider. <https://docs.microsoft.com/en-us/azure/quantum/provider-honeywell>.
- [92] Microsoft. [n.d.]. Azure Quantum. <https://azure.microsoft.com/en-us/services/quantum/>.
- [93] Ashley Montanaro. 2016. Quantum algorithms: An overview. *npj Quantum Information* 2, 1 (2016), 1–8.
- [94] W. J. Munro, K. A. Harrison, A. M. Stephens, S. J. Devitt, and Kae Nemoto. 2010. From quantum multiplexing to high-performance quantum networking. *Nature Photonics* 4, 11 (2010), 792.
- [95] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1015–1029.
- [96] Prakash Murali, Norbert Matthias Linke, Margaret Martonosi, Ali Javadi Abhari, Nhung Hong Nguyen, and Cinthia Huerta Alderete. 2019. Full-stack, real-system quantum computer studies: Architectural comparisons and design insights. In *Proceedings of the 46th International Symposium on Computer Architecture*. 527–540.
- [97] Prakash Murali, David C. McKay, Margaret Martonosi, and Ali Javadi-Abhari. 2020. Software mitigation of crosstalk on noisy intermediate-scale quantum computers. *arXiv preprint arXiv:2001.02826* (2020).
- [98] Michael A. Nielsen and Isaac Chuang. 2002. Quantum computation and quantum information.
- [99] Jeremy L. O’Brien, Akira Furusawa, and Jelena Vučković. 2009. Photonic quantum technologies. *Nature Photonics* 3, 12 (2009), 687.
- [100] Giuseppe Davide Paparo and M. A. Martin-Delgado. 2012. Google in a quantum network. *Scientific Reports* 2 (2012), 444.
- [101] Daniel K. Park, June-Koo K. Rhee, and Soonchil Lee. 2018. Noise-tolerant parity learning with one quantum bit. *Physical Review A* 97, 3 (2018), 032327.
- [102] Raj B. Patel, Joseph Ho, Franck Ferreyrol, Timothy C. Ralph, and Geoff J. Pryde. 2016. A quantum Fredkin gate. *Science Advances* 2, 3 (2016), e1501531.
- [103] Tirthak Patel, Abhay Potharaju, Baolin Li, Rohan Roy, and Devesh Tiwari. 2020. Experimental evaluation of NISQ quantum computers: Error measurement, characterization, and implications. In *2020 SC20: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE Computer Society, 636–650.
- [104] Jarryd J. Pla, Kuan Y. Tan, Juan P. Dehollain, Wee H. Lim, John J. L. Morton, David N. Jamieson, Andrew S. Dzurak, and Andrea Morello. 2012. A single-atom electron spin qubit in silicon. *Nature* 489, 7417 (2012), 541–545.
- [105] John Preskill. 2018. Quantum computing in the NISQ era and beyond. *Quantum* 2 (2018), 79.
- [106] Timothy J. Proctor, Arnaud Carignan-Dugas, Kenneth Rudinger, Erik Nielsen, Robin Blume-Kohout, and Kevin Young. 2019. Direct randomized benchmarking for multiqubit devices. *Physical Review Letters* 123, 3 (2019), 030503.
- [107] Quantiki. 2020. List of QC simulators. <https://www.quantiki.org/wiki/list-qc-simulators>.
- [108] Yihui Quek, Stanislav Fort, and Hui Khoo Ng. 2021. Adaptive quantum state tomography with neural networks. *npj Quantum Information* 7, 1 (2021), 1–7.
- [109] Patrick Rebentrost, Brajesh Gupta, and Thomas R. Bromley. 2018. Quantum computational finance: Monte Carlo pricing of financial derivatives. *Physical Review A* 98, 2 (2018), 022321.
- [110] Rigetti. [n.d.]. Native gates for Rigetti QPUs. <https://pyquil-docs.rigetti.com/en/v2.7.0/apidocs/gates.html>.
- [111] Chad Rigetti, Jay M. Gambetta, Stefano Poletto, B. L. T. Plourde, Jerry M. Chow, A. D. Córcoles, John A. Smolin, Seth T. Merkel, J. R. Rozen, George A. Keefe, et al. 2012. Superconducting qubit in a waveguide cavity with a coherence time approaching 0.1 ms. *Physical Review B* 86, 10 (2012), 100506.
- [112] Gonçalo Sampaio. 2017. Code in QASM for quantum circuits and algorithms. <https://github.com/sampaio96/Quantum-Computing>.
- [113] Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. 2015. An introduction to quantum machine learning. *Contemporary Physics* 56, 2 (2015), 172–185.
- [114] Yunong Shi, Nelson Leung, Pranav Gokhale, Zane Rossi, David I. Schuster, Henry Hoffmann, and Frederic T. Chong. 2019. Optimized compilation of aggregated instructions for realistic quantum computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1031–1044.

- [115] Peter W. Shor. 1994. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE, 124–134.
- [116] Peter W. Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review* 41, 2 (1999), 303–332.
- [117] Daniel R. Simon. 1997. On the power of quantum computation. *SIAM Journal on Computing* 26, 5 (1997), 1474–1483.
- [118] Kaitlin N. Smith and Mitchell A. Thornton. 2019. A quantum computational compiler and design tool for technology-specific targets. In *Proceedings of the 46th International Symposium on Computer Architecture*. 579–588.
- [119] Damian S. Steiger, Thomas Häner, and Matthias Troyer. 2018. ProjectQ: An open source software framework for quantum computing. *Quantum* 2 (2018), 49.
- [120] Samuel Stein, Nathan Wiebe, Yufei Ding, Peng Bo, Karol Kowalski, Nathan Baker, James Ang, and Ang Li. 2022. EQC: Ensembled quantum computing for variational quantum algorithms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 59–71.
- [121] Samuel A. Stein, Betis Baheri, Daniel Chen, Ying Mao, Qiang Guan, Ang Li, Shuai Xu, and Caiwen Ding. 2022. QuClassi: A hybrid deep neural network architecture based on quantum state fidelity. *Proceedings of Machine Learning and Systems* 4 (2022), 251–264.
- [122] Samuel A. Stein, Betis Baheri, Ray Marie Tischio, Ying Mao, Qiang Guan, Ang Li, Bo Fang, and Shuai Xu. 2020. QuGAN: A generative adversarial network through quantum states. *arXiv preprint arXiv:2010.09036* (2020).
- [123] Juexiao Su, Tianheng Tu, and Lei He. 2016. A quantum annealing approach for Boolean satisfiability problem. In *2016 53rd ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [124] Swamit S. Tannu and Moinuddin Qureshi. 2019. Ensemble of diverse mappings: Improving reliability of quantum computers by orchestrating dissimilar mistakes. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 253–265.
- [125] Swamit S. Tannu and Moinuddin K. Qureshi. 2019. Mitigating measurement errors in quantum computers by exploiting state-dependent bias. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 279–290.
- [126] Swamit S. Tannu and Moinuddin K. Qureshi. 2019. Not all qubits are created equal: A case for variability-aware policies for NISQ-era quantum computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 987–999.
- [127] Tommaso Toffoli. 1980. Reversible computing. In *International Colloquium on Automata, Languages, and Programming*. Springer, 632–644.
- [128] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N. Smith, Joshua Vizslai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R. Martonosi, and Frederic T. Chong. 2022. SupermarQ: A scalable quantum benchmark suite. *arXiv preprint arXiv:2202.11045* (2022).
- [129] Vlatko Vedral, Adriano Barenco, and Artur Ekert. 1996. Quantum networks for elementary arithmetic operations. *Physical Review A* 54, 1 (1996), 147.
- [130] Salvador Elías Venegas-Andraca. 2012. Quantum walks: A comprehensive review. *Quantum Information Processing* 11, 5 (2012), 1015–1106.
- [131] Harald Weinfurter. 1994. Experimental bell-state analysis. *EPL (Europhysics Letters)* 25, 8 (1994), 559.
- [132] James D. Whitfield, Jacob Biamonte, and Alán Aspuru-Guzik. 2011. Simulation of electronic structure Hamiltonians using quantum computers. *Molecular Physics* 109, 5 (2011), 735–750.
- [133] Michael Wilde, Mihael Hategan, Justin M. Wozniak, Ben Clifford, Daniel S. Katz, and Ian Foster. 2011. Swift: A language for distributed parallel scripting. *Parallel Comput.* 37, 9 (2011), 633–652.
- [134] Stefan Woerner and Daniel J. Egger. 2019. Quantum risk analysis. *npj Quantum Information* 5, 1 (2019), 1–8.
- [135] Muqing Zheng, Ang Li, Tamás Terlaky, and Xiu Yang. 2020. A Bayesian approach for characterizing and mitigating gate and measurement errors. *arXiv preprint arXiv:2010.09188* (2020).
- [136] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D. Lukin. 2018. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *arXiv preprint arXiv:1812.01041* (2018).

Received 1 June 2021; revised 7 April 2022; accepted 10 July 2022